Git Essentials – 2

Branches, Merge, Rebase, Cherry-Pick, Stash & Debugging with Git

1. What Is a Git Branch Really?

Suppose the commit history is:

```
A ── B ── C
```

If `main` is the current branch, it essentially points to commit `C`:

```
A ── B ── C
            ↑
          main
```

A Git branch is simply a **lightweight, movable reference to a commit**.

It does not contain a separate copy of the project. It is essentially a name that resolves to a particular commit.

> **Mental model:** `main` is fundamentally a name pointing to a commit.

Inspecting a Branch Reference

Git stores branch references inside:

```
ls .git/refs/heads
```

You may see:

```
main
```

To inspect what `main` currently points to:

```
cat .git/refs/heads/main
```

Output will be a commit object ID such as:

```
81ca253...
```

That hash is the commit currently referenced by `main`.

---

2. What Is `HEAD`?

Inspect `HEAD`:

```
cat .git/HEAD
```

In a normal attached state, you will usually see something like:

```
ref: refs/heads/main
```

Conceptually:

```
HEAD
 ↓
main
 ↓
Commit C
```

Or more simply:

```
HEAD → main → C
```

`HEAD` tells Git which branch or commit you are currently working on.

---

3. What Happens to a Branch When You Commit?

Suppose the history is:

```
A — B — C
        ↑
      main
        ↑
      HEAD
```

Now create a new commit `D`.

After the commit:

```
A — B — C — D
            ↑
          main
            ↑
          HEAD
```

In the normal attached state, `HEAD` still points to `main`.

What actually moves is the branch reference:

```
main: C → D
```

So the relationship becomes:

```
HEAD → main → D
```

---

4. Creating and Switching Branches

Create a new branch:

```
git branch feature-login
```

Now both branches can point to the same commit:

```
            main
              ↓
A — B — C — D
              ↑
         feature-login
```

Nothing large is copied. Git simply creates another reference.

Switch to the feature branch:

```
git switch feature-login
```

Now:

```
HEAD
 ↓
feature-login
 ↓
D
```

If you create a new commit `E`, only `feature-login` moves:

```
A — B — C — D — E
            ↑       ↑
          main   feature-login
                    ↑
                  HEAD
```

This is the foundation behind branching, merging and rebasing.

---

5. Making the Feature Branch Move Independently

Suppose `feature-login` gets additional commits while `main` stays unchanged:

```
              main
                ↓
A ── B ── C
              \
               D ── E
                     ↑
              feature-login
                     ↑
                   HEAD
```

Why did `main` not move?

Because:

```
HEAD → feature-login
```

New commits move the branch referenced by `HEAD`.

---

6. Viewing Git History as a Graph

A very useful command for understanding branches is:

```
git log --oneline --graph --decorate --all
```

Options

- `-oneline` → compact commit history
- `-graph` → visualizes branches and merges
- `-decorate` → shows branch and reference names
- `-all` → includes commits reachable from all refs

Before making destructive Git changes, inspecting the graph is often a good first step.

---

7. Fast-Forward Merge

Suppose `main` has not changed after the feature branch was created:

```
HEAD
 ↓
main
 ↓
C
 \
  D — E
      ↑
 feature-login
```

Switch to `main`:

```
git switch main
```

Merge the feature branch:

```
git merge feature-login
```

Because `main` is an ancestor of `feature-login`, Git can simply move the `main` reference forward.

Before

```
A — B — C
        \
         D — E
```

After

```
A — B — C — D — E
                ↑
               main
                ↑
               HEAD
```

The feature branch may also still point to `E`.

This is called a **fast-forward merge**.

> Git does not need to create a merge commit when the current branch can simply move forward to the target branch.

---

8. When Fast-Forward Is Not Possible

Create another feature branch:

```
git switch -c feature-payment
```

Suppose the feature branch gets commit `D`:

```
         D  ← feature-payment
        /
A — B — C
```

Now switch back to `main`:

```
git switch main
```

Suppose `main` independently gets commit `E`:

```
         D  ← feature-payment
        /
A — B — C
        \
         E  ← main
```

The branches have now **diverged**.

Git cannot simply move `main` to `D`, because commit `E` also exists on `main`.

The two histories must be combined.

---

9. Three-Way Merge

Run:

```
git switch main
git merge feature-payment
```

If the changes can be combined without conflicts, Git creates a merge commit.

Conceptually:

```
         D
        / \
A — B — C   M
        \ /
         E
```

A normal commit usually has one parent:

```
normal commit
→ one parent
```

A merge commit commonly has two parents:

```
merge commit
→ two parents
```

You can inspect the merge commit using:

```
git cat-file -p HEAD
```

You may see:

```
tree ...
parent <commit-E>
parent <commit-D>
author ...
committer ...
```

```
Merge branch 'feature-payment'
```

Why Is It Called a Three-Way Merge?

Git considers three important snapshots:

1. **Common ancestor**
2. **Current branch (`ours`)**
3. **Branch being merged (`theirs`)**

Example:

```
          Feature D
         /
Ancestor C
         \
          Main E
```

Git compares:

```
C → D
```

and:

```
C → E
```

This allows Git to understand which changes happened independently on each branch.

10. Merge Conflicts

A merge conflict happens when Git cannot safely decide how competing changes should be combined.

A conflicted file can contain markers such as:

```
<<<<<<<
=======
>>>>>>>
```

Resolve the file manually by keeping the correct final content and removing the conflict markers.

Then stage the resolved file:

```
git add config.txt
```

Complete the merge:

```
git commit
```

---

11. Why Does Rebase Exist?

Suppose the history looks like this:

```
A ── B ── C ── F ── G
            \
             D ── E
```

Where:

```
main    → G
feature → E
```

A normal merge is completely valid:

```
A – B – C – F – G ──── M
        \           /
         D ──────── E
```

However, in a large project with many developers and frequent merges, the graph can become difficult to read:

```
*   Merge feature-12
|\
| * commit
| * commit
* | main commit
|/
*   Merge feature-11
|\
| * commit
...
```

Rebase provides another way to integrate changes while producing a more linear history.

---

12. Rebase from First Principles

Start with:

```
A — B — C — F — G
        \
         D — E
```

While on the feature branch, run:

```
git rebase main
```

Conceptually, Git:

1. Finds commits unique to the feature branch: `D` and `E`
2. Temporarily sets their changes aside
3. Moves the feature branch base to `G`
4. Replays the changes introduced by `D` and `E`
5. Creates new commits `D'` and `E'`

Result:

```
A — B — C — F — G — D' — E'
                        \
                         feature
```

More conceptually:

```
main → G
feature → E'
```

The important point is:

```
D ≠ D'
E ≠ E'
```

Rebase does not simply move the old commits. It creates new commits by replaying their changes on top of another base.

---

13. Why Do Rebased Commits Get New IDs?

A Git commit contains information such as:

```
Tree
Parent
Author
Committer
Timestamp
Message
```

Before rebase:

```
D
parent → C
```

After replaying it on top of `G`:

```
D'
parent → G
```

Because the commit metadata is now different, the commit hash changes.

Therefore:

```
D ≠ D'
```

The same is true for `E` and `E'`.

This is why rebase is described as **history rewriting**.

---

14. Merge vs Rebase

Merge

- Preserves the existing branch topology
- Does not rewrite existing commits
- Usually safer for shared history
- Can introduce additional merge commits

Rebase

- Produces a cleaner, more linear history
- Replays commits on top of a new base
- Changes commit identities
- Rewrites history

> **Practical rule:** Rebase your own local or unshared work freely. Be very careful rebasing history that other developers have already based their work on.

If a teammate already has commit `D`, but you rewrite it into `D'`, Git sees them as different commits:

```
D != D'
```

The histories no longer align naturally.

---

15. Cherry-Pick

Sometimes you do not want an entire branch. You only want the change introduced by one specific commit.

Suppose:

```
feature-payment:

A — B — C — D — E
```

Commit `D` contains an important security fix, but commit `E` contains unfinished payment work.

Production branch:

```
A — B — F
```

You only want the change introduced by `D`.

Run:

```
git cherry-pick <commit-D>
```

Result:

```
A — B — F — D'
```

Again:

```
D' != D
```

Git applies the change in a different history/context and creates a new commit.

Cherry-Pick Mental Model

```
Commit D
   ↓
Find what changed between
parent(D) → D
   ↓
Apply that change on the current branch
   ↓
Create D'
```

Common Use Cases

- Hotfixes
- Applying one specific bug fix
- Taking one useful commit from another branch
- Backporting a fix

---

16. Git Stash

Suppose you are working on a payment feature and have modified:

```
PaymentService.java
PaymentController.java
```

The work is incomplete.

Now an urgent production issue appears and you need to switch to `main`.

Running:

```
git switch main
```

may be unsafe if your local changes would be overwritten, or Git may carry those changes to the other branch when that is not what you want.

The work is also not ready for a meaningful commit.

This is where `git stash` is useful.

Saving Work Temporarily

Run:

```
git stash
```

Mental model:

```
Working changes
     ↓
Temporarily saved by Git
     ↓
Working tree cleaned
```

Now you can switch branches:

```
git switch main
```

After fixing the production issue, return:

```
git switch feature-payment
```

View stored stashes:

```
git stash list
```

Example:

```
stash@{0}: WIP on feature-payment...
```

Restore the latest stash:

```
git stash pop
```

`git stash apply` vs `git stash pop`

Apply

```
git stash apply
```

- Restores the stash
- Keeps the stash entry

Pop

```
git stash pop
```

- Restores the stash
- Removes the stash entry if the application succeeds

---

17. Git as a Debugging and History Investigation Tool

Git is not only for storing code history. It can also help investigate production issues.

Imagine:

- The application worked last Monday
- Today it is broken

- 150 commits happened in between

You may want to know:

- Who changed something?
- When was it changed?
- What exactly changed?
- Which commit introduced the bug?
- Can an older state be recovered?

Git history can answer these questions.

---

18. `git log`

Basic command:

```
git log
```

Compact history:

```
git log --oneline
```

Example:

```
78fab32 Fix payment timeout
638af11 Add payment validation
a328b19 Add JWT support
91acf21 Initial commit
```

Graph view:

```
git log --oneline --graph --decorate --all
```

Example:

```
*   b92fc2d (HEAD -> main) Merge feature-payment
|\
| * 39df839 Add payment API
| * 728ab22 Add payment validation
* | 64ae212 Fix login issue
|/
* 12bd882 Initial commit
```

This is especially useful before performing history-changing or destructive operations.

---

19. `git show`

Suppose commit `39df839` looks suspicious.

Run:

```
git show 39df839
```

`git show` gives you the commit metadata along with the patch introduced by that commit.

It helps answer questions such as:

```
Who made the commit?
When was it made?
What was the commit message?
Which files changed?
Which lines changed?
```

This makes it useful during debugging and incident investigation.

---

20. `git diff` Between Commits or Branches

Compare two commits:

```
git diff <commit-A> <commit-B>
```

Example:

```
git diff a328b19 78fab32
```

This answers:

> What changed between these two points in history?

You can also compare branches:

```
git diff main feature-payment
```

This is useful during code review and debugging.

---

21. `git bisect`

Suppose the history is:

```
A — B — C — D — E — F — G — H
```

You know:

```
A = application working
H = application broken
```

You want to find the **first commit that introduced the bug**.

Checking every commit one by one is inefficient, especially if hundreds or thousands of commits exist.

`git bisect` solves this using a binary-search-style approach.

Binary Search Intuition

Instead of testing commits sequentially:

```
A
B
C
D
E
F
G
H
```

Git checks a commit near the middle.

Suppose Git checks `D`.

```
A ✅ → B → C → [D ?] → E → F → G → H ❌
```

If `D` works:

```
git bisect good
```

Git now knows the bug cannot be in `A`, `B`, `C`, or `D`.

The search is reduced to:

```
D ✅ → E → F → G → H ❌
```

Git may next check `F`.

If `F` is broken:

```
git bisect bad
```

Now Git knows:

```
D = GOOD
F = BAD
```

So the problem must have appeared between them:

```
D ✅ → E ? → F ❌
```

Git checks `E`.

If `E` is also broken:

```
git bisect bad
```

Now:

```
D ✅ → E ❌
```

Therefore `E` is the first bad commit.

Git may report:

```
e41c972 is the first bad commit
```

Meaning:

> The application worked before `e41c972`, and starting from that commit the bad behavior appeared.

---

22. How to Use `git bisect`

Suppose:

```
A → B → C → D → E → F → G
```

You are currently on `G`, and the application is broken.

Commit `A` is known to be working.

Step 1: Start bisect mode

```
git bisect start
```

Step 2: Mark the current commit as bad

```
git bisect bad
```

Equivalent to:

```
git bisect bad HEAD
```

Step 3: Mark a known working commit as good

```
git bisect good <commit-id-of-A>
```

Example:

```
git bisect good 12ab45
```

Git automatically checks out a commit roughly in the middle:

```
A ✅ → B → C → [D ?] → E → F → G ❌
```

You do not manually choose `D`. Git chooses it.

Step 4: Test the application

If `D` works:

```
git bisect good
```

Git may now check `F`:

```
A ✅ → B → C → D ✅ → E → [F ?] → G ❌
```

Test again.

If the checked commit is broken:

```
git bisect bad
```

Continue marking each tested commit as `good` or `bad`.

Eventually Git narrows the range to a single commit and reports something like:

```
abc123 is the first bad commit
```

It may also display the commit details:

```
commit abc123...
Author: Aditya
Date: ...

    Added payment validation logic
```

You have now identified the commit that introduced the regression.

Ending Bisect Mode

After finishing:

```
git bisect reset
```

During a bisect session Git moves through historical commits.

`git bisect reset` ends bisect mode and returns you to the state you were in before starting.

---

23. Why `git bisect` Is Useful in DevOps

`git bisect` can help locate the commit where behavior changed in situations such as:

- Application startup failure
- Docker image suddenly becoming too large
- Health endpoint failure
- Tests starting to fail
- Increased latency
- Configuration no longer loading
- Build pipeline failure

The key requirement is that you can classify tested commits as:

```
GOOD
or
BAD
```

Git can then keep narrowing the range.

---

24. Automating `git bisect`

If you already have an automated test command such as:

```
./mvnw test
```

or:

```
npm test
```

Git can execute a script automatically while performing the bisect.

Mental model:

```
Git chooses commit
       ↓
```

```
Run automated test
       ↓
Exit 0 → good
Failure → bad
       ↓
Git chooses next commit
```

Command:

```
git bisect run <script>
```

This connects Git debugging directly with automated testing and CI/CD thinking.

Quick Command Reference

Task	Command
Create a branch	`git branch <branch-name>`
Switch branch	`git switch <branch-name>`
Create and switch branch	`git switch -c <branch-name>`
Visualize branch history	`git log --oneline --graph --decorate --all`
Merge a branch	`git merge <branch-name>`
Rebase current branch	`git rebase <branch-name>`
Apply one commit	`git cherry-pick <commit-id>`
Temporarily save changes	`git stash`
View stashes	`git stash list`
Restore and keep stash	`git stash apply`
Restore and remove stash	`git stash pop`
Show a commit	`git show <commit-id>`
Compare commits/branches	`git diff <A> <B>`
Start bisect	`git bisect start`
Mark commit good	`git bisect good`
Mark commit bad	`git bisect bad`

Task	Command
End bisect	`git bisect reset`
Automate bisect	`git bisect run <script>`